```
end

triangular_numbers.lazy.map { _1.first }.select(&:even?).take(5).to_a # => [6,
                                                                       # .. 10,
                                                                       # .. 28,
                                                                       # .. 36,
                                                                       # .. 66]
```

An enumerator can be created for a series of enumerables with the product method, which takes one or more enumerable objects as arguments and returns a new Enumerator that is made up of the Cartesian product of all the arguments:

```
product = Enumerator.product("a".."c", -1..1, [:x, :y])
product.to_a # => [["a", -1, :x], ["a", -1, :y], ["a", 0, :x], ["a", 0, :y],
             # .. ["a", 1, :x], ["a", 1, :y], ["b", -1, :x], ["b", -1, :y], ["b",
             # .. 0, :x], ["b", 0, :y], ["b", 1, :x], ["b", 1, :y], ["c", -1,
             # .. :x], ["c", -1, :y], ["c", 0, :x], ["c", 0, :y], ["c", 1, :x],
             # .. ["c", 1, :y]]
```

Using Enumerators

There are a few things you can do with an enumerator. You can cause it to move though its values externally by calling next. This will move through any chained logic, but is not associated with a block, so it won't invoke a block. The rewind method puts the enumerator back to the beginning of the sequence.

You can also use peek to look at the next value in the enumerator without moving the iterator forward—successive calls to peek will return the same object:

```
x = [1, 2, 3].to_enum.with_index
x.next    # => [1, 0]
x.next    # => [2, 1]
x.peek    # => [3, 2]
x.peek    # => [3, 2]
x.rewind  # => #<Enumerator: #<Enumerator: [1, 2, 3]:each>:with_index>
x.next    # => [1, 0]
```

As shown in this example, enumerators can be chained together. The with_index method chains the iterator to a new iterator that includes the index of each object as a second argument to the block. The related method with_object takes an object (usually a container like an array or a hash) and creates a new enumerator that passes that object as a second argument.

You can also use + to add another enumerable to the end of the current enumerator, appending those values to the end of what's already there.

Using each with a block causes the entire enumerator to be invoked. The each method also takes arguments that might have needed to be passed to earlier parts of the enumerator. In this case, the string is converted to an enumerator based on the split method, but the each call passes the argument that determines what the string is split on:

```
x = "a string of words".to_enum(:split)
x.each("o").to_a  # => ["a string ", "f w", "rds"]
```